# Архитектура

Английский оригинал: ARCHITECTURE.md.

## Подход

Проект — локальное настольное приложение из трёх взаимодействующих частей:

1. **Движок захвата** (`src/capture/`) — читает файл PCAP/PCAPNG и строит нормализованный захват: TCP-сессии, собранные направленные потоки и провенанс каждого байта.
2. **Движок протокола и гипотез** (`src/protocol/`, `src/hypothesis/`) — разбивает направленный поток на сообщения, применяет декларативные правила интерпретации, классифицирует результаты, проверяет правила на корпусе, находит контрпримеры и версионирует правила и результаты.
3. **Проект, отчёт и интерфейс** — `src/project/` (переносное исследование), `src/report/` (отрисовка отчёта) и `src/ui/` (интерфейс на PySide6).

Интерфейс между частями зафиксирован JSON-контрактами в `docs/CONTRACT.md` и схемами в `docs/schemas/`. Ни одна часть не заглядывает в модель данных другой; части обмениваются контрактным JSON и обычными словарями.

Каждый модуль определяют три принципа:

- **Наблюдение отделено от гипотезы.** Движок захвата сообщает байты и их происхождение. Он никогда не говорит, что байт означает.
- **Контрпримеры обязательны.** Правило, совпавшее с примерами, отчитывается как подтверждённое в проверенном объёме, а противоречащие ему данные сохраняются.
- **Воспроизводимость.** При одном и том же входе каждый шаг даёт один и тот же выход, а проект хранит относительные пути и контрольные суммы, поэтому открывается без изменений на другой машине.

## Движок захвата

### Выбор библиотеки

Чтение PCAP и PCAPNG выполняется через **dpkt 1.9.8**.

- dpkt читает и классический pcap (`dpkt.pcap.Reader`), и pcapng (`dpkt.pcapng.Reader`) без внешних процессов.
- PyShark отклонён: он запускает `tshark`, который должен быть установлен на целевой машине, а эталонное окружение этого не гарантирует.
- scapy держится как мысленный запасной вариант, но зависимостью не является: он тяжелее и медленнее для эталонной нагрузки до 250k пакетов.

IPv6 и несколько типов канального уровня (raw IP, Linux cooked capture, Linux loopback, BSD cooked) разбираются; неподдерживаемый тип канального уровня или испорченный заголовок сообщается как диагностика, а не выбрасывается исключением, поэтому смешанный захват всё равно даёт читаемые TCP-сессии.

### Модули

- `parser.py` — `read_capture(path) -> Iterator[Packet]`. Выдаёт TCP-пакеты с монотонным `index`, временной меткой, конечными точками, номерами последовательности, флагами и нагрузкой. Не-TCP пакеты пропускаются с диагностикой, где это уместно. Неподдерживаемый тип канального уровня сообщается как диагностика; IPv6, включая цепочку extension-заголовков, разбирается. Фрагментированная датаграмма передаётся в `fragment.py` и даёт один пакет, когда все её фрагменты собраны. Повреждённый, но распознаваемый файл читается насколько возможно, остаток сообщается диагностиками; исключение выбрасывает только не-захват.
- `fragment.py` — `FragmentReassembler` собирает фрагменты IPv4 и IPv6 и возвращает датаграмму, когда присутствует каждый байт. Для IPv4 при перекрытии сохраняется первое прибытие и оно сообщается; для IPv6 датаграмма отбрасывается при любом перекрытии, как требует RFC 5722. Датаграмма, которая так и не собралась, сообщается как `ip_fragment_incomplete`, и ни один байт не домысливается.
- `session.py` — `build_sessions(packets) -> list[Session]`. Группирует пакеты в экземпляры TCP-соединений. Новая сессия начинается на SYN без ACK, на новой четвёрке адрес/порт после FIN/RST и на любом повторном использовании пары конечных точек после завершения предыдущего экземпляра.
- `reassembly.py` — `reassemble(session, direction, packets) -> DirectionalStream`. Упорядочивает нагрузку по номеру последовательности, дедуплицирует одинаковые ретрансмиссии, буферизует сегменты, пришедшие не по порядку, явно помечает пропуски, записывает противоречивые перекрытия как неоднозначность и обрабатывает переход через границу 32-битного номера последовательности.
- `provenance.py` — `Range` и помощники, отображающие смещения потока на исходные пакеты.
- `export.py` — `export_capture(...)` пишет контрактный JSON потоком, поэтому весь файл не держится в памяти.
- `export_wireshark.py` — пишет по одному пакету на направление сессии с собранными байтами, для просмотра в Wireshark.
- `streaming.py` — `process_streaming(...)` освобождает закрытую сессию из памяти во время обхода большого захвата.
- `cli.py` — `python -m src.capture.cli --pcap ... --out ...`. `--verbose` пишет ход чтения, сессий и экспорта в stderr; без него команда печатает только JSON-сводку.

## Проект и отчёт

- `src/project/manifest.py` — модель манифеста на диске и его проверка.
- `src/project/project.py` — `Project`: создание, открытие, `add_capture`, `export`, `import_`. Каждый путь в манифесте относительный, а каждый захват несёт свой sha256, поэтому каталог можно переместить и снова открыть; импорт отвергает подделанный архив.
- `src/report/markdown.py` и `src/report/html.py` — отрисовывают словарь исследования в отчёт Markdown и в самодостаточную HTML-страницу, оформленную по `docs/STYLE.md`.
- `src/report/model.py` — форма словаря исследования, общая для обоих отрисовщиков и создаваемая стадией протокола.

## Форматы

См. `docs/CONTRACT.md`. Форматов три:

1. **Нормализованный захват** — выход движка захвата, вход движка протокола и интерфейса.

2. **Результат применения правила** — выход движка протокола, вход интерфейса.
3. **Манифест проекта** — индекс исследования на диске.

## Зависимости

| Пакет | Версия | Зачем |
|---|---|---|
| `dpkt` | 1.9.8 | Читает pcap и pcapng на чистом Python, без внешнего процесса. |
| `PyYAML` | 6.0.2 | Правила интерпретации можно писать и как YAML, и как JSON. |
| `PySide6` | 6.8.1 | Настольный интерфейс. |
| `pytest` | 8.3.4 | Тесты (дополнение для разработки). |
| `jsonschema` | 4.23.0 | Проверяет выгрузки по схемам (дополнение для разработки). |

## Ограничения и что не реализовано

- **IPv6** разбирается. TCP-сессии поверх IPv6 дают потоки как для IPv4, а контрольные суммы TCP для IPv6 проверяются с `--verify-checksums`. Фрагментация IPv6 собирается; перекрытие отбрасывает всю датаграмму, как требует RFC 5722, и конфликт сообщается.
- **Фрагментация IP** собирается для обеих версий. Фрагменты одной датаграммы могут быть разделены посторонним трафиком и доставлены не по порядку. Перекрытие сообщается для IPv4 (сохраняется первое прибытие) и отбрасывает датаграмму для IPv6. Датаграмма, которая так и не собралась, сообщается как `ip_fragment_incomplete`; её байты не домысливаются.
- **Зашифрованный трафик** не расшифровывается. TLS и всё остальное поверх TCP предстаёт непрозрачными байтами; инструменты уровня байт работают и по нему.
- **Понимаются только Ethernet, Linux loopback и BSD cooked** типы канального уровня. Остальные сообщаются как `unsupported_linktype` и пропускаются.
- **Проверка контрольных сумм по умолчанию выключена** из-за checksum offloading на отправляющем узле; её можно включить, когда известно, что захват цел.
- **Конфликтующие перекрытия не разрешаются.** Сохраняются первые байты, а противоречие сообщается как `ambiguity`, вместо молчаливого выбора одной версии.

## Производительность

Измеренная эталонная нагрузка — в `docs/BENCHMARK.md`: 95.66 МиБ входа, 250k пакетов, 1000 сессий, за 11.56 с при пике памяти 354 МиБ и за 11.22 с при 224 МиБ в потоковом режиме, что даёт 122.44 МиБ JSON. Оба значения с большим запасом укладываются в лимиты задания: 5 минут и 4 ГиБ.